

Attention & Transformers

Moin Mostakim

What is a Transformer?

- A Transformer is a sequence model that relies on **attention only**, without recurrence (RNN) or convolution (CNN).
- Core idea: **self-attention** lets each token weigh all other tokens and build a context-aware representation.
- Originally introduced for machine translation (source sentence $\rightarrow$ target sentence).

Historical context

Attention started in neural machine translation (2014), then became the center of the Transformer architecture (2017, “Attention Is All You Need”).

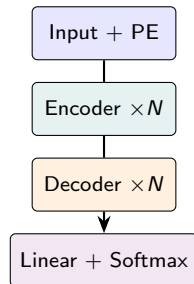
Why attention matters

- Attention assigns importance weights across tokens while encoding a token.
- It solves fixed-length bottlenecks of older seq2seq systems by dynamically focusing on relevant input parts.
- Intuition: each token asks, *Which other tokens matter for me, and by how much?*

Year	Architecture	Role of attention
2014	RNN encoder–decoder	Soft alignment
2015	RNN + attention	Better translation focus
2017	Transformer	Attention-only backbone

Transformer at a glance

- **Encoder stack:** repeated blocks of
 - Multi-head self-attention
 - Feed-forward network (FFN)
 - Residual + layer norm around each sublayer
- **Decoder stack:** masked self-attention + cross-attention + FFN, with residual + layer norm.
- Typical original depth: 6 encoders and 6 decoders.

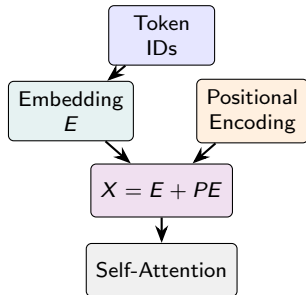


Why positional encoding is required

- Attention processes tokens in parallel; by itself it does **not** encode order.
- Without positions, many permutations can become ambiguous.
- Positional encoding (PE) is added to embeddings before the first attention layer.

Desired PE properties

Unique per position, consistent across lengths, supports relative reasoning, deterministic, and generalizable.



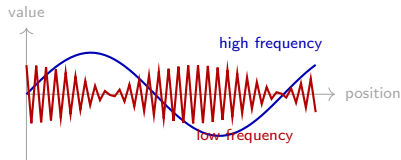
Sinusoidal positional encoding

For token position pos , model dimension d_{model} , and channel index i :

$$PE(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right),$$

$$PE(\text{pos}, 2i + 1) = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right).$$

- Even channels: sine, odd channels: cosine.
- Frequency changes with i , capturing short and long positional patterns.
- Deterministic formula supports longer sequences at inference.

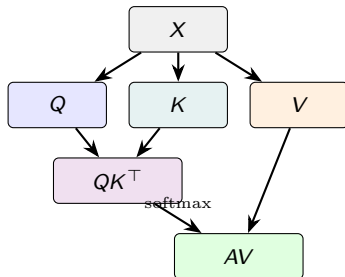


Self-attention intuition: Q, K, V

- From embeddings X :

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

- Q**: what this token asks for.
- K**: what each token offers for matching.
- V**: information to be mixed by attention weights.



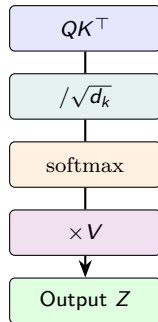
Context effect

Scores come from query-key dot products; weighted sum of V gives context-aware output.

Scaled dot-product attention (steps)

- 1 Compute scores: $S = QK^{\top}$
- 2 Scale: $S/\sqrt{d_k}$
- 3 Normalize: $A = \text{softmax}(S/\sqrt{d_k})$
- 4 Weight values: $Z = AV$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right) V$$

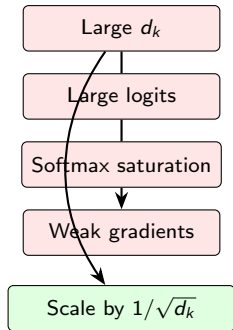


Why divide by $\sqrt{d_k}$?

- Dot product: $x = q \cdot k = \sum_{i=1}^{d_k} q_i k_i$.
- As d_k increases, $\text{Var}(x)$ increases.
- Large logits saturate softmax, causing tiny gradients and unstable learning.

$$\text{Var}\left(\frac{x}{f(d_k)}\right) \propto \frac{d_k}{f(d_k)^2}$$

Choosing near-constant variance gives $f(d_k) = \sqrt{d_k}$.

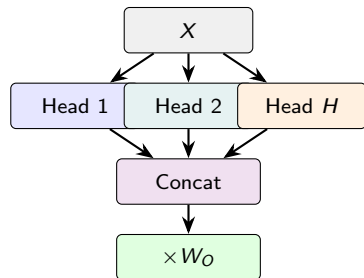


Multi-head attention

- One head may over-focus on one relation type.
- Multiple heads learn different relation subspaces in parallel.
- Each head uses separate $W_Q^{(h)}$, $W_K^{(h)}$, $W_V^{(h)}$:

$$\text{head}_h = \text{Attention}(Q_h, K_h, V_h)$$

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_H) W_O$$

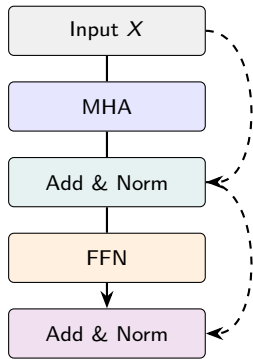


Encoder block (compact)

- 1 Multi-head self-attention
- 2 Add & Norm (residual + layer norm)
- 3 Position-wise FFN
- 4 Add & Norm again

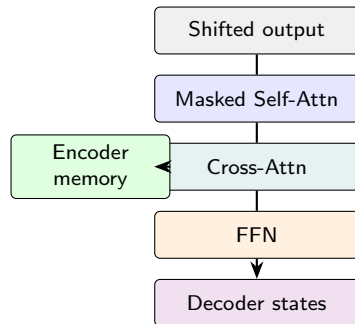
Role split

Attention models token interactions globally;
FFN adds nonlinear transformation locally per token.



Decoder block (compact)

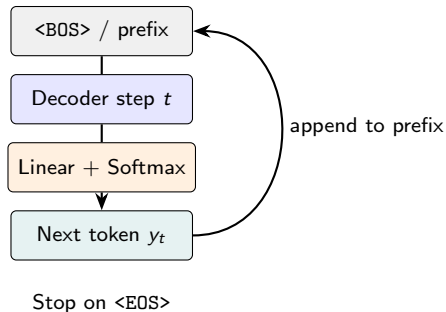
- 1 **Masked** self-attention on output prefix
- 2 Cross-attention (Q from decoder, K/V from encoder)
- 3 FFN, each with Add & Norm wrappers
 - Autoregressive token-by-token decoding.
 - Mask blocks future-token leakage.
 - Cross-attention aligns output with source.



Autoregressive generation loop

- Start with <BOS> token.
- Decoder returns hidden state h_t .
- Linear + softmax produce token distribution.
- Pick next token and feed it back.
- Stop when <EOS> is generated.

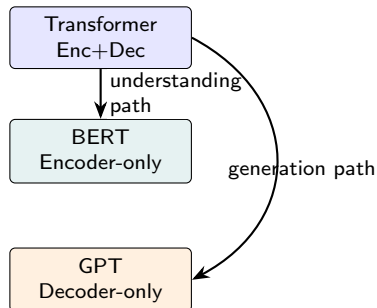
$$p(y_t \mid y_{<t}, x) = \text{softmax}(W_o h_t + b_o)$$



Transformer vs BERT vs GPT

Model	Core stack	Direction	Typical usage
Transformer	Enc+Dec	Bidir + causal	Seq2Seq
BERT	Encoder-only	Bidirectional	NLU tasks
GPT	Decoder-only	Causal	Generation

- BERT objective: MLM (historically also NSP).
- GPT objective: causal language modeling.



Key takeaways

- Attention replaces recurrence with fully parallel token interaction.
- Positional encoding restores order information.
- Scaled dot-product + multi-head design gives stable and expressive context modeling.
- Encoder captures source context; decoder generates autoregressively with masking and cross-attention.
- Final linear + softmax maps decoder states to words.

Practice math (attention)

Q1. Scaled dot-product attention (single query, 2 keys).

Let $q = [1, 2]$, $k_1 = [1, 0]$, $k_2 = [0, 2]$, $v_1 = [2, 1]$, $v_2 = [0, 3]$, and $d_k = 2$.

- 1 Compute raw scores $s_i = q \cdot k_i$.
- 2 Scale by $\sqrt{d_k}$, apply softmax to get α_1, α_2 .
- 3 Compute output $z = \alpha_1 v_1 + \alpha_2 v_2$.

Q2. Why $\sqrt{d_k}$?

If $\text{Var}(q \cdot k) \propto d_k$, show that scaling by $f(d_k)$ gives

$$\text{Var}\left(\frac{q \cdot k}{f(d_k)}\right) \propto \frac{d_k}{f(d_k)^2},$$

and derive the $f(d_k)$ that keeps variance approximately constant.

Q3. Multi-head dimensions check.

If $d_{\text{model}} = 512$ and heads $H = 8$:

- What is per-head d_k ?
- Why does concatenating 8 heads recover width 512 before multiplying by W_O ?

References I

- Vaswani et al., *Attention Is All You Need*, 2017.
- Illustrated Transformer (Jay Alammar): <https://jalammar.github.io/illustrated-transformer/>
- Transformers laid out: https://goyalpramod.github.io/blogs/Transformers_laid_out/
- Additional visual note: <https://x.com/viplismism/status/2003807608571076782>